IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 1 August 2025, accepted 17 August 2025, date of publication 20 August 2025, date of current version 26 August 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3600678

RESEARCH ARTICLE

Efficient Bearing Fault Diagnosis for Edge Computing Using Grayscale Spectrograms and Hybrid Neural Model Compression

HUYNH-ANH-HUY NGUYEN AND CHEOL HONG KIM
School of Computer Science and Engineering, Soongsil University, Seoul 06978, South Korea

Corresponding author: Cheol Hong Kim (cheolhong@ssu.ac.kr)

This work was supported by the National Research Foundation of Korea (NRF) Grant through Korean Government [Ministry of Science and ICT (MSIT)] under Grant RS-2024-00453676.

ABSTRACT Bearing elements play important roles in mechanical machining. Efficient monitoring and detection of bearing faults are critical for ensuring motor reliability. Deep learning techniques, particularly those that use convolutional neural networks, have emerged as prominent solutions for achieving high accuracy in bearing fault classification. However, the computational complexities of these models render them unsuitable for use in resource-constrained devices. In this study, we explored the application of a combination of multiple techniques, including knowledge distillation, layer fusion, and quantization-aware training, to develop lightweight models that accelerate inference speed while maintaining high accuracy. In addition, acoustic emission signals containing bearing health information were preprocessed using a short-time Fourier transform and converted into grayscale spectrograms before being fed into a neural network. The experimental results indicate that the proposed method can significantly reduce the inference latency and memory footprint while achieving high accuracy for different bearing fault types at variable rotational speeds.

INDEX TERMS Acoustic emission, ball bearings, condition monitoring, convolutional neural networks, digital signal processing, deep learning, edge AI, fault diagnosis, rotating machines, spectrogram.

I. INTRODUCTION

Industrial machinery serves as the backbone of modern manufacturing and production processes, in which numerous rotating and reciprocating components collaborate to ensure smooth operation. Among these components, bearings play an indispensable role in supporting mechanical rotation, reducing friction, and extending equipment lifespan. However, bearing faults are the most common cause of severe machinery failures, accounting for 50% of such incidents [1], leading to unplanned downtimes and substantial financial losses if not promptly detected and addressed [2]. Therefore, the early detection and accurate diagnosis of bearing faults are of paramount importance. Historically, manual inspections have served as the primary line of defense, relying on visual checks, audible noise evaluations, and periodic maintenance schedules, which can be time-consuming and susceptible to human error. Although these conventional methods can uncover certain faults, they often hinge on human expertise and may overlook subtle early stage defects. Consequently, automated diagnostic systems are increasingly being utilized to bridge these gaps by employing sensor-driven data acquisition and signal-processing techniques. Vibration signals have traditionally dominated this field owing to well-established analytical methods and strong track records in fault detection [3], [4]. These signals mostly lie in the lower-frequency ranges and capture the global resonant response of a structure or rotating element. By collecting and analyzing vibration data from sensors mounted on or near bearings, engineers and researchers can monitor subtle changes in the signal characteristics that often precede a full-scale malfunction. The ability to detect incipient defects such as spalls on the inner

The associate editor coordinating the review of this manuscript and approving it for publication was Gerard-Andre Capolino.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

or outer race, damaged rolling elements, or cage instabilities, makes vibration-based monitoring an essential component of condition-based maintenance strategies. Despite the reliability of vibration-based methods, their real-world implementations encounter significant noise and interference. Vibrations from gearboxes, shafts, and other adjacent rotating components can mask or distort specific patterns associated with the bearings. Environmental factors such as temperature fluctuations, dust, and moisture can further alter the sensor output. Overcoming these challenges requires careful sensor placement, robust signal-processing filters, and adaptive thresholding techniques that adjust to changing operational conditions. In certain instances, sophisticated denoising and feature-extraction algorithms are required to prevent false alarms and missed detections [5].

Acoustic emission (AE) signals are gaining traction because of their nonintrusive nature and capacity to highlight distinctive bearing fault patterns, particularly in environments where vibration measurement is more challenging. AE signals are high-frequency stress waves generated by the rapid release of strain energy within a material or machine component, which typically occurs during crack initiation, crack propagation, or other defect-related phenomena [6]. This high-frequency domain is often less contaminated by ambient machinery or environmental noise which predominantly clusters at lower frequencies. Tang et al. concluded that AE signals exhibit a relatively high signal-to-noise ratio compared with vibration signals [7]. Consequently, AE signals can provide a clearer signature for diagnostic purposes, thereby reducing the complexity of filtering and feature extraction. Furthermore, AE-based systems can be designed for near-real-time monitoring because transient events are detected and processed quickly after they occur. Although advanced data-acquisition electronics and high-speed sampling are typically necessary, modern digital hardware has made it feasible to implement real-time algorithms that can recognize AE bursts almost instantaneously.

With the proliferation of artificial intelligence (AI), which encompasses classical machine learning techniques, such as support vector machines (SVMs) and random forests, as well as state-of-the-art deep learning paradigms, such as convolutional neural networks (CNNs), its applicability has permeated a diverse array of disciplines. Nguyen and Lee proposed a nonlinear equalizer using a CNN for holographic data storage systems and achieved superior accuracy compared with traditional equalizers [8]. Bui et al. used Cloud-Net and Change-Net for change detection in satellite-based Earth observation applications [9]. In the field of fault diagnosis, deep learning techniques have been widely utilized as extensions of data-driven approaches [10], [11]. A CNN model can automatically discover patterns in AE signals that correlate with specific fault types or severity levels. These models can be generalized beyond the conditions under which they are trained to identify subtle early stage faults and issue timely warnings by leveraging large training datasets spanning diverse operating conditions. Anbalagan et al. proposed a physics-informed deep learning approach for bearing fault detection under different motor working conditions [12]. Wan et al. used an improved 2D LeNet-5 network to address the problems of low recognition accuracy, slow convergence speed, and weak generalization ability associated with traditional LeNet-5 [13].

However, a notable limitation of CNN-based methods in the field of bearing fault diagnosis is the high computational resource overhead compared with common traditional methods [14]. This issue has been investigated in several studies. These studies have employed custom lightweight CNNs tailored for edge computing or embedded devices with constrained resources. However, most of these investigations have concentrated on a single aspect or a specific type of technique, failing to comprehensively address the multifaceted challenges encountered in real-world scenarios. First, many compression methods have been validated on controlled datasets (e.g., CWRU). However, they degrade under real-world conditions characterized by noise, nonstationary speeds, or imbalanced fault distributions. Second, although a few studies have incorporated compression techniques, they continue to impose substantial computational demands during the data-processing phase. Pham et al. applied the pruning technique to MobileNetV2 at various scaling rates to demonstrate its feasibility and reliability for use in embedded systems targeted at bearing fault diagnosis. However, they did not address critical aspects such as the dataset memory footprint or the complexity associated with the data-processing technique [15]. Pham et al. also utilized a neural architecture search and shared weights to achieve a significant reduction in the number of multiply-accumulate (MAC) operations, parameters, and overall inference latency. However, this approach remains constrained by its high power consumption, memory usage, and computationally intensive data-processing flow [16]. Furthermore, benchmarks often neglect deployment metrics such as the energy consumption and latency of microcontrollers, favoring theoretical compression ratios. Yang et al. developed an improved knowledge-distillation method to address the problems of low computational accuracy and challenging hardware deployment in practical engineering applications of rolling bearing fault diagnosis [17]. Despite its promising design, the lack of demonstrated deployment in embedded systems raises concerns regarding its real-world feasibility.

An alternative approach to lightweight classification involves the use of one-dimensional CNNs (1D-CNNs) [18]. Although 1D-CNNs offer reduced model complexity, they are generally less effective for handling nonstationary AE signals and tend to underperform in terms of classification accuracy compared with 2D-CNNs. Binarized neural networks (BNNs) are a promising approach for real-time inference in edge computing scenarios. They replace expensive floating-point multiplications with simple bitwise operations such as XNOR and bit counting, leading to substantial gains

in inference speed and energy efficiency [19]. Regardless of these benefits, BNNs often experience a decrease in accuracy compared with their full-precision counterparts, particularly when applied to complex tasks or datasets. Another active research trend is stochastic computing (SC), a computation paradigm that represents and processes information using random bit streams, where numerical values are encoded as probabilities, allowing complex arithmetic operations to be performed using simple logic gates such as AND, OR, and multiplexers [20], [21]. This approach significantly reduces the power consumption and circuit area, making it highly attractive for low-power and fault-tolerant systems. Despite these strengths, SC suffers from limitations, including long computation times owing to the need for extended bit streams to achieve acceptable precision. Spiking neural networks (SNNs) have emerged as the next generation of artificial neural networks that encode and transmit information through spikes, mimicking the behavior of the human brain [22], [23]. These event-driven processing models lead to extremely sparse and efficient computations; however, they depend on neuromorphic hardware to achieve low-power consumption and remain a major challenge in training frameworks because of their nondifferentiable nature. A more recent advancement in this domain is the adoption of hardware-aware neural architecture search (HW-NAS), which automatically designs CNN architectures optimized not only for high accuracy but also for accelerating the inference speed, reducing memory usage, and lowering the number of floating-point operations, enabling deployment on various target platforms, especially ultra-low-resource microcontrollers [24], [25], [26], [27]. Despite these advantages, the computational complexity of search algorithms is a barrier to their practical adoption, particularly in scenarios requiring rapid model deployment and design cycles.

Although recent advances in bearing fault diagnosis for edge deployment have made notable progress, significant limitations remain in addressing the diverse operational complexities encountered in real-world applications. Many studies have focused on model compression techniques, such as network pruning, quantization, and architecture optimization, to reduce computational complexity. However, these efforts often overlook the crucial aspects of robust and efficient data preprocessing. Furthermore, many of the proposed methods have not been rigorously validated through actual deployment in embedded systems, instead relying on evaluations conducted in desktop-based environments. This disconnection between algorithmic development and practical implementation raises concerns regarding the true readiness of current approaches for real-world industrial applications, particularly in highly resource-constrained edge scenarios.

To address these challenges, this study emphasizes the optimization of both the signal-processing and inference stages of a bearing fault diagnosis framework that leverages 2D single-channel spectrograms derived from AE signals in conjunction with hybrid compression techniques to achieve a balance between diagnostic accuracy and computational efficiency. The proposed approach is designed for edge AI environments and demonstrates the potential for real-time operation on central processing units (CPUs) rather than relying on graphics processing units (GPUs). Specifically, a short-time Fourier transform (STFT) is employed to analyze the bearing signals at variable rotational speeds. This enables a time–frequency representation of AE signals, effectively capturing transient and nonstationary features that are critical for accurate fault diagnosis, while offering a well-balanced trade-off between diagnostic accuracy and computational efficiency. To further optimize data processing, we convert the three-channel spectrograms into single-channel grayscale representations. This dimension reduction significantly decreases the storage requirements of the dataset while maintaining high classification accuracy. Consequently, the number of input channels in the CNN architecture is reduced, resulting in a lighter inference pipeline.

In addition, we incorporated several model optimization strategies to minimize the memory footprint and enhance the inference speed. First, knowledge distillation (KD) was employed to transfer learned representations from a high-capacity teacher model to a compact student model. By aligning the output logits or intermediate feature distributions, KD enables the student model to approximate the teacher's performance while significantly reducing the number of parameters and overall computational cost. Second, layer fusion is applied to combine adjacent linear operations, such as convolution, followed by batch normalization and activation, into a single computational unit. This transformation simplifies the execution graph, minimizes the memory access overhead, and reduces the inference latency, particularly for hardware accelerators and embedded platforms. Third, we adopted quantization-aware training (QAT) to prepare a model for low-bit inference while maintaining accuracy. Unlike post-training quantization, QAT simulates the quantization effects during training, allowing the model to adapt its weights. As a result, the model achieves substantial reductions in model size and inference time with minimal performance loss. The main contributions of this study are as follows:

1) The experimental testbed is specifically designed to collect AE signals directly, thereby eliminating the need for extensive denoising procedures that are commonly required when acquiring vibration signals.
2) We employ an efficient STFT to convert raw AE signals into 2D spectrograms, enabling the time–frequency representation of nonstationary signal characteristics. To validate the effectiveness of our preprocessing strategy, we conducted a comparative analysis against other widely used time–frequency analysis techniques. Subsequently, the generated spectrograms were transformed into single-channel grayscale images, which significantly reduced both computational complexity and dataset storage requirements. This transformation streamlines the input pipeline of the model while

preserving the essential discriminative features necessary for accurate fault classification.

3) We integrate multiple compression techniques rather than relying on a single approach to leverage their collective advantages. A high-performance teacher model guides the training of a significantly smaller student model by using the KD technique. This process enables the development of a compact model with performance comparable to that of its larger counterparts. We also implemented layer fusion, which helps decrease computation overhead. A QAT method is utilized to reduce the model size and further enhance the inference efficiency while preserving high accuracy.
4) The final model is executed on an embedded device to assess the proposed method under practical conditions.

The remainder of this paper is organized as follows. Section II reviews related studies on data-processing and model-compression techniques. Section III elaborates on the proposed method and details its underlying procedures and theoretical foundations. In Section IV, the experimental method is presented, followed by a presentation and analysis of the results. Finally, Section V provides conclusions and discusses potential directions for future research.

II. RELATED WORK

Fourier transform (FT) is a basic digital signal-processing (DSP) technique that converts time-domain signals into frequency-domain representations. A primary advantage of FT is its ability to decompose complex AE signals into discrete frequency components and efficiently isolate them, enabling precise fault identification [28]. Furthermore, the computational efficiency of the fast Fourier transform (FFT) algorithm facilitates real-time monitoring, which is essential for industrial applications requiring immediate fault detection. Despite its strengths, FT faces limitations when analyzing nonstationary AE signals because their frequency content evolves over time.

Other methods can offer enhanced time–frequency analysis and improve transient fault detection. STFT is an extension of FT. It applies FT to segmented portions of a signal using a sliding time window to generate a time–frequency spectrogram [29]. Wavelet transform (WT) uses mother wavelets of localized waveforms with adjustable scales to decompose signals into multiresolution components, thereby addressing the shortcomings of STFT [30]. Hilbert–Huang transform (HHT) combines empirical mode decomposition (EMD) and Hilbert transform to analyze nonlinear and nonstationary signals. EMD adaptively decomposes AE signals into intrinsic mode functions (IMF), from which the instantaneous frequencies are derived [31]. HHT is particularly effective for diagnosing faults under variable loads or speeds; however, it is susceptible to mode mixing and noise sensitivity [32].

Similarly, an advanced DSP technique, known as the constant Q transform (CQT), was developed to analyze signals using a frequency resolution that geometrically adapts and aligns with the logarithmic spacing of musical scales. Unlike STFT, which employs uniform frequency bins, CQT ensures that the ratio of the center frequency to the bandwidth (Q factor) remains constant across all frequency bins [33], [34]. However, although mitigated by recent algorithms, CQT consistently requires more computational resources than STFT [35], which poses challenges in resource-constrained environments. A comparison of STFT, CQT, and HHT is summarized in Table 1.

TABLE 1. Comparative analysis of representative time-frequency processing methods concerning edge deployment constraints.

	STFT	CQT	HHT
Computational Complexity	Moderate	High	Very high
Memory Usage	Moderate	High	Very high
Power Consumption	Moderate	High	Very high
Frequency Resolution	Fixed resolution	Variable resolution	Adaptive resolution
Use Case	Good for edge computing	Not ideal for edge computing	Not suitable for edge computing

In addition to these techniques, recent advancements have focused on transforming 1-D time-series data into 2-D image representations to leverage the powerful feature extraction capabilities of CNNs. For example, Lian et al. proposed a novel time-series data-imaging scheme that converts raw vibration signals into images using various DSP methods [36], thereby enhancing diagnostic performance by capturing features across multiple domains within a unified image representation. Similarly, frameworks such as CFI-LFENet utilize cross-domain fusion imaging with lightweight feature-enhanced networks [37], achieving high diagnostic accuracy in identifying various fault types while reducing the computational complexity.

Beyond transform-based methods, recent research has increasingly focused on stationary–nonstationary disentanglement techniques to improve the interpretability of fault diagnosis models. These methods aim to separate signals into components that capture long-term invariant patterns (stationary features) and those that reflect dynamic fault-related variations (nonstationary features). Although traditional signal decomposition methods have been used for this purpose, more recent studies have leveraged deep learning-based representations for improved performance and flexibility. Yu et al. [38] proposed a model for industrial nonstationary fault detection using multiscale content learning to disentangle features in the data. Yu et al. [39] introduced a mutual stacked autoencoder architecture to enhance local information and address the challenges of multivariable fault detection in complex industrial environments.

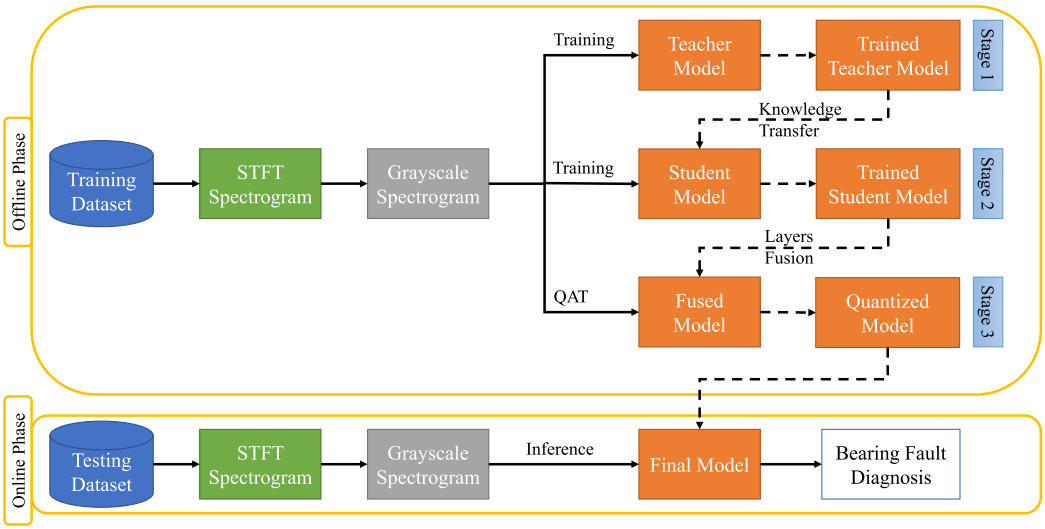


FIGURE 1. Workflow of the proposed bearing fault diagnosis method. In the offline phase, the model is trained and compressed through three stages. In the online phase, preprocessed grayscale spectrograms are fed into the optimized model for real-time fault classification.

Neural network compression has emerged as a critical research area for addressing the computational, memory, and energy constraints associated with the deployment of deep learning models on resource-limited devices. Pruning involves removing redundant parameters while preserving model performance. Early work by Han et al. popularized magnitude-based weight pruning, which iteratively trims weights with minimal absolute values [40]. Subsequent studies introduced structured pruning, which removes all neurons, filters, and channels to maintain hardware-friendly sparsity patterns [41], [42].

Ding et al. utilized adaptive pruning to avoid the repetitive and cumbersome processes characteristic of traditional pruning techniques for bearing fault diagnosis [43]. Quantization helps reduce the numerical precision of the weights and activations. Binary [44] and ternary networks [45] achieve extreme compression, but often experience accuracy degradation. Li et al. applied k-means quantization to reduce the computational cost of the model significantly while ensuring unchanged accuracy [46]. To address the performance degradation in low-bit regimes, Chung et al. proposed a QAT combined with post-training quantization to improve the training convergence speed and minimize the number of parameters [47]. Another compression technique is KD, which transforms knowledge from complex models into lightweight variants. Ma et al. used a hybrid model framework that combines KD and deep forests for fault classification [48]. Despite this progress, key limitations hinder industrial adoption. Therefore, data processing and neural model compression techniques must be considered to address these barriers.

III. PROPOSED METHOD

This section outlines the proposed bearing fault diagnosis method that applies different compression techniques to deploy advanced deep learning models on embedded devices to achieve real-time inference in resource-constrained environments. The methodology is structured into two phases: offline and online. Fig. 1 shows a diagram of the proposed method.

A. OVERALL PROCEDURE

This section describes the general process of our efficient bearing fault diagnosis method for edge devices, including the AE signal acquisition, data preprocessing, model compression, testing, and visualization of the results. The procedure is summarized as follows:

Step 1: Set up a testbed to acquire AE signals containing bearing health information in different states under various working conditions.

Step 2: Segment the acquired AE signals using a fixed-length window (0.05 s) [49]. Apply STFT to each segment to create a spectrogram image.

Step 3: Convert each STFT spectrogram into a single-channel grayscale image.

Step 4: Choose the teacher (ResNet-50) and student (MobileNetV2 and MobileNetV3-L) models. Modify the first layer of these models from a 3-channel input to a single-channel input. Train the teacher model until it converges. Use the KD from the trained teacher model to train the student models.

Step 5: Fuse the convolution, batch normalization, and rectified linear unit (ReLU) layers in the trained student models.

Step 6: Apply QAT to the fused models to obtain quantized models.

Step 7: Measure the performance and other metrics of the final model (accuracy, memory footprint, and inference time) in two environments: desktop and embedded device.

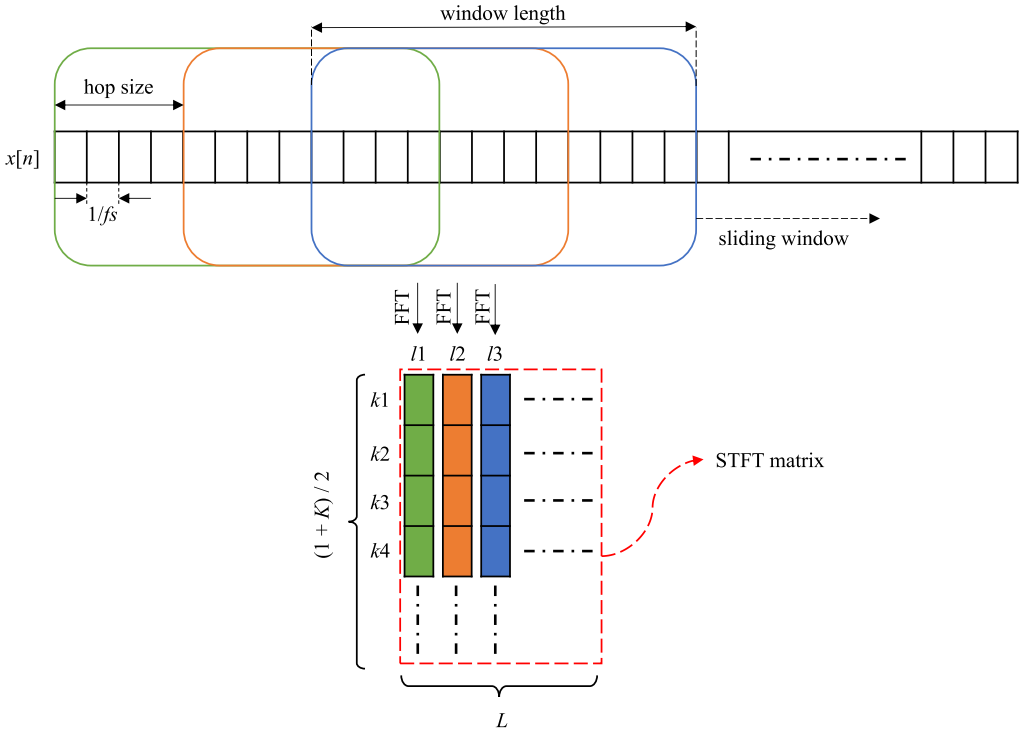


FIGURE 2. Processing procedure for applying the Short-Time Fourier Transform (STFT). This process captures the time-frequency representation of the signal by analyzing localized frequency content over time. The STFT matrix is obtained by sliding a Blackman window across the analyzed signal with a stride equal to the hop size, computing the DFT at each step.

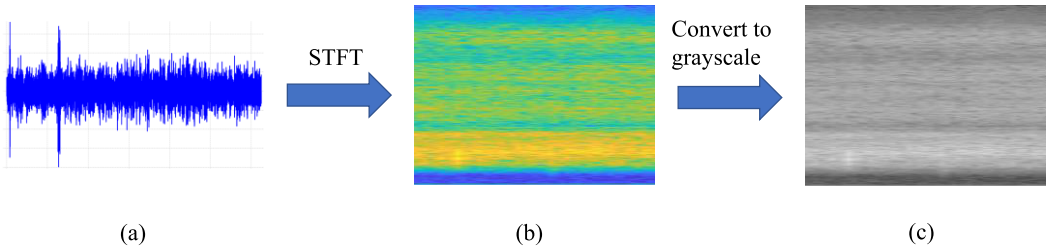


FIGURE 3. AE signal transformation and spectrogram generation process: (a) Segmented AE signal extracted from the raw data, (b) STFT spectrogram representing the time-frequency distribution, (c) Converted grayscale spectrogram.

Step 8: Analyze and evaluate the results and compare them to other methods.

B. DATA PREPROCESSING

The first approach involves converting time-domain signals into a time–frequency representation using STFT. By segmenting the raw signal into overlapping frames and applying a suitable window function to each frame, STFT captures the local frequency information over time. Mathematically, the STFT process for a discrete-time signal can be represented as follows:

$$x_l[m] = x[m + lH]\, w[m] \tag{1}$$

$$X[k, l] = \frac{1}{M} \sum_{m=1}^{K} x_l[m] e^{-\frac{j2\pi mk}{K}} \tag{2}$$

where $m \in \{1, 2, \ldots, M\}$ is an index related to the start of the sliding window, $M \in \mathbb{N}$ is the window length, $H \in \mathbb{N}$ is the hop size, $l \in \{0, 1, \ldots, L-1\}$ is the frame index, $L \in \mathbb{N}$ is the number of frames, $k \in \{1, 2, \ldots, K\}$ is the frequency bin index, and $K \in \mathbb{N}$ is the number of discrete Fourier transform (DFT) points.

The matrix $X[k, l]$ is the STFT of $x[n]$ (Fig. 2). Each frame l of M samples is selected from the analyzed signal and then is multiplied by the Blackman window $w[m]$. Each result forms a column in the STFT matrix, with the frequency increasing

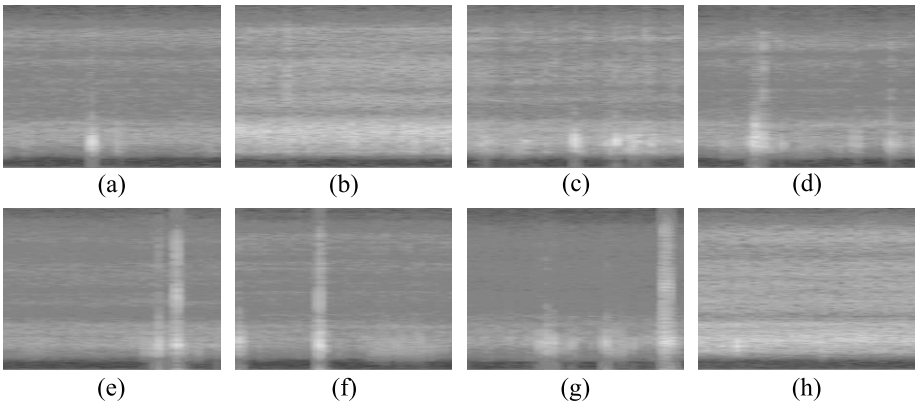


FIGURE 4. Representative grayscale spectrograms of 7 fault cases and 1 normal case: (a) inner fault (BCI), (b) outer fault (BCO), (c) roller fault (BCR), (d) inner and outer fault (BCIO), (e) outer and roller fault (BCOR), (f) inner and roller fault (BCIR), (g) inner-outer-roller fault (BCIOR), (h) normal condition (BNC).

down the rows. Every row of the matrix comprises the DFT coefficients associated with a specific frequency over time. The magnitude of $X[k, l]$ serves as a two-dimensional representation of the spectral energy distribution over time, and its spectrogram visualization is implemented as

$$\tilde{X}[k, l] = 20 \log \left(\frac{2}{\sum_{m=1}^{M} \frac{w[m]}{M}} |X[k, l]| + 10^{-6} \right) \quad (3)$$

After analyzing and referring to numerous experiments [50], we chose a window length (M) of 1024, hop size of $M/4$, and 4096 DFT points. The STFT spectrograms were used as visual images to indicate the amplitudes of the frequencies in both the time and frequency domains. The process of AE signal-processing was applied for seven kinds of single and compound-bearing faults (inner (BCI), outer (BCO), roller (BCR), inner and outer (BCIO), outer and roller (BCOR), inner and roller (BCIR), inner-outer-roller (BCIOR)), and normal condition (BNC) at various rotational speeds (250, 300, 350, 400, 450, and 500 RPM).

Converting STFT results into a grayscale spectrogram offers a concise, visually intuitive representation of time–frequency data that highlights critical amplitude variations without unnecessary color complexity. This single-channel format not only reduces the computational overhead for downstream processing, but also simplifies data preparation. Figs. 3 and 4 show the AE signal-processing flow and grayscale spectrograms, respectively. Furthermore, by forgoing color information, which often does not affect diagnostic performance, analysts and algorithms can more readily discern subtle shifts in the signal energy. Ultimately, grayscale spectrograms strike an effective balance between preserving essential structural information and maintaining a streamlined vision, which is both computationally efficient and diagnostically potent.

C. KNOWLEDGE DISTILLATION

KD is a technique introduced to transfer the knowledge learned by a large, complex model (often referred to as the teacher) to a smaller, more efficient model (the student). This is a type of transfer-learning method. First popularized by Hinton et al., KD has become a powerful methodology for compressing deep neural networks without a substantial loss in predictive accuracy [51]. Intuitively, the teacher model output softens the target distribution for the student, guiding it to learn both the correct labels and interclass relationships, which are often captured in teacher-output probabilities even for incorrect classes. By exploiting these softened probabilities, the student model can achieve a better generalization than if it is trained purely on hard one-hot labels.

One critical advantage of KD is that the student model can have far fewer parameters than the teacher model while maintaining a comparable level of accuracy. This process is akin to model compression; however, rather than simply pruning weights or applying quantization, the method leverages richer "dark knowledge" in the teacher's output distribution. This subtle interclass information acts as a form of regularization, helping students to generalize better than from purely hard labels.

Let $z^T \in \mathbb{R}^K$ be the teacher logits, which are the raw outputs for K classes for an input x. The teacher probability distribution $p^T(x)$ at a temperature $\tau > 1$ is then defined by

$$p_i^T(x; \tau) = \frac{\exp\left(\frac{z_i^T}{\tau}\right)}{\sum_{j=1}^{K} \exp\left(\frac{z_j^T}{\tau}\right)} \quad (4)$$

where z_i^T is the i-th logit of the teacher for class i. A higher temperature τ produces a smoother, less peaky distribution, illuminating relative probabilities across all classes rather than focusing solely on the most likely label. Similarly, let $z^S \in \mathbb{R}^K$ be the student logits for the same input x. The

student distribution $p^S(x)$ under temperature τ is

$$p_i^S(x;\tau) = \frac{\exp\left(\frac{z_i^S}{\tau}\right)}{\sum_{j=1}^{K}\exp\left(\frac{z_j^S}{\tau}\right)} \tag{5}$$

KD typically involves a combined loss function that incorporates both ground-truth labels and softened teacher distributions. One common objective ($\mathcal{L}_{total}$) is the weighted sum of two terms: the standard cross-entropy ($\mathcal{L}_{CE}$) and Kullback–Leibler divergence ($\mathcal{L}_{KD}$):

$$\mathcal{L}_{CE}\left(y, p^S(x;1)\right) = -\sum_{i=1}^{K} y_i \log(p_i^S(x;1)) \tag{6}$$

$$\mathcal{L}_{KD}\left(p^T(x;\tau), p^S(x;\tau)\right) = \tau^2 \sum_{i=1}^{K} p_i^T(x;\tau)\log\left(\frac{p_i^T(x;\tau)}{p_i^S(x;\tau)}\right) \tag{7}$$

where $p^S(x;1)$ denotes the student distribution at $\tau = 1$. The factor τ^2 compensates for the gradient scaling effect introduced by higher temperatures.

Combining these yields the complete KD loss:

$$\mathcal{L}_{total} = \alpha\mathcal{L}_{CE}\left(y, p^S(x;1)\right) + (1-\alpha)\mathcal{L}_{KD}\left(p^T(x;\tau), p^S(x;\tau)\right) \tag{8}$$

where $\alpha \in [0,1]$ is a hyperparameter balancing the emphasis on hard-label loss versus distillation loss from the teacher. If α is close to 1, the student is trained primarily to match the ground truth, with only a small contribution from the teacher's soft targets. Conversely, if α is close to zero, the student is guided more strongly by the teacher's distribution. A moderate value of α often yields the best trade-off in practice. Therefore, in this study, the α and τ are set to 0.5 and 3, respectively.

D. LAYER FUSION

Layer fusion in deep neural networks commonly refers to the merging of multiple sequential operations such as convolution (Conv), batch normalization (BN), and ReLU into a single computational block. This technique, often called ''operator fusion,'' is especially useful for boosting inference speed, reducing memory overhead, and simplifying deployment on resource-constrained hardware. Although each of these layers serves a distinct purpose during training, at the inference time, they can be mathematically combined without altering the overall functionality of the model. By minimizing redundant reads and writes in the memory, fused operations can exploit modern hardware more efficiently and yield performance gains.

Consider a standard 2D convolution (with stride and padding omitted for simplicity); let $x \in \mathbb{R}^{C_{in}\times H\times W}$ be the input feature map, and let $W \in \mathbb{R}^{C_{out}\times C_{in}\times k_h\times k_w}$ and $b \in \mathbb{R}^{C_{out}}$ denote the convolution weights and biases, respectively. An output $c \in \mathbb{R}^{C_{out}\times H'\times W'}$ will be

$$c_{o,h,w} = \sum_{i=1}^{C_{in}}\sum_{r=1}^{k_h}\sum_{s=1}^{k_w} W_{o,i,r,s}x_{i,h+r-1,w+s-1} + b_o \tag{9}$$

where o indicates the output channel, and (h,w) indicates the spatial position in the output feature map.

Immediately after convolution, many architectures apply BN to stabilize the training and improve the generalization. For the output channel o, BN is defined as

$$y_{o,h,w} = \gamma_o\frac{c_{o,h,w}-\mu_o}{\sqrt{\sigma_o^2+\epsilon}} + \beta_o \tag{10}$$

where μ_o and σ_o^2 are the running mean and variance for channel o. γ_o and β_o are learnable affine parameters, and ϵ is a small constant for numerical stability. During training, μ_o and σ_o^2 are typically estimated from mini-batches. However, during inference, these values are fixed, allowing them to be merged with the weights and biases of the convolution.

A key insight for inference-time fusion is that the scaling and shifting introduced by the BN can be folded into an equivalent set of convolution parameters. Specifically, we can define the effective weights W' and effective biases b' such that the convolution alone produces a BN-adjusted output:

$$y_{o,h,w} = \alpha_o c_{o,h,w} + \beta_o' \tag{11}$$

where

$$\alpha_o = \frac{\gamma_o}{\sqrt{\sigma_o^2+\epsilon}} \quad \text{and} \quad \beta_o' = \beta_o - \frac{\gamma_o\mu_o}{\sqrt{\sigma_o^2+\epsilon}}$$

By substituting into (9), the output of the fused convolution inherently incorporates the BN effect:

$$c'_{o,h,w} = \sum_{i=1}^{C_{in}}\sum_{r=1}^{k_h}\sum_{s=1}^{k_w}\alpha_o W_{o,i,r,s}x_{i,h+r-1,w+s-1} + \alpha_o b_o + \beta_o' \tag{12}$$

Mathematically, Conv+BN becomes an equivalent convolution with the adjusted parameters, thereby eliminating the BN layer during inference. ReLU is then either implemented as a separate activation or, in some frameworks, fused further into the same kernel for Conv+BN+ReLU. The final step is effectively transformed as follows:

$$z = \max(0,\ c'_{o,h,w}) \tag{13}$$

E. QUANTIZATION-AWARE TRAINING

QAT is an advanced technique that enables deep neural networks to learn robust representations with limited numerical precision. Unlike post-training quantization, which quantizes a pre trained model without adjusting its weights, QAT incorporates a quantization process during the training phase. This integration enables the model to adjust its parameters according to precision constraints, thereby mitigating the decrease in accuracy typically observed in naïve quantization

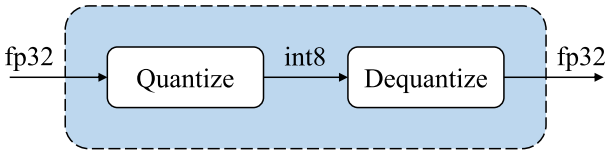


FIGURE 5. Fake quantization module including two steps: quantization and dequantization. This process enables the network to simulate quantization effects during training, allowing it to adapt and thus preserve accuracy when deployed on resource-constrained hardware.

approaches. QAT is particularly crucial for deploying models on resource-constrained devices, such as mobile phones, microcontrollers, or specialized hardware accelerators, where memory and computation must be tightly optimized.

Fake quantization in the forward pass is a technique used in QAT to simulate the effect of low-precision arithmetic on model weights and activation during training at floating point. This helps the network learn parameters that are robust to the discrete low-bit representation used at inference time. During training, it is impractical to completely replace the floating-point operations with integer operations. Thus, we injected a fake quantization module to approximate integer quantization; however, all actual computations, such as matrix multiplication and convolution, were still executed with floating-point precision. The module includes two steps (Fig. 5) that convert the floating-point value into an integer-like representation, and then convert it back to a pseudo-floating-point value, including a quantization error.

Each layer's weights and activations go through a "quantize → dequantize" cycle during the forward pass. For instance, let x be a floating-point tensor that we wish to falsely quantize. We define a scale $\alpha > 0$, which determines how big each quantized step is, and a zero-point z to handle the asymmetric ranges. The integer range of 8-bit values would be $[q_{min}, q_{max}]$. The common fake quantization function can be expressed as follows:

$$Q(x) = \alpha(\min\left(\max\left(\left\lfloor \frac{x}{\alpha} + z + 0.5 \right\rfloor, q_{\min}\right), q_{\max}\right) - z) \tag{14}$$

We also inserted observers into each layer to monitor the distribution of the activations or weights. They collect statistics such as min/max values during the forward pass of training, which helps determine a good scale α and zero point z for quantization. Because of the nondifferentiable function (14), such as rounding and clipping in the fake quantization module (Fig. 6), the training workflow is typically broken by zeroing the gradients. Thus, during the backpropagation pass, the gradient is computed as if the quantization function is the identity, which is called the straight-through estimator (STE). Specifically, the partial derivatives of the clip and round operators are set to 1 inside the valid range and 0 outside to allow the gradients to flow through the quantization as if they were mostly transparent. After the model converges, we remove all fake quantization modules and convert the model into a real quantized int8 network to perform the inference. In summary, the QAT workflow is implemented as follows.

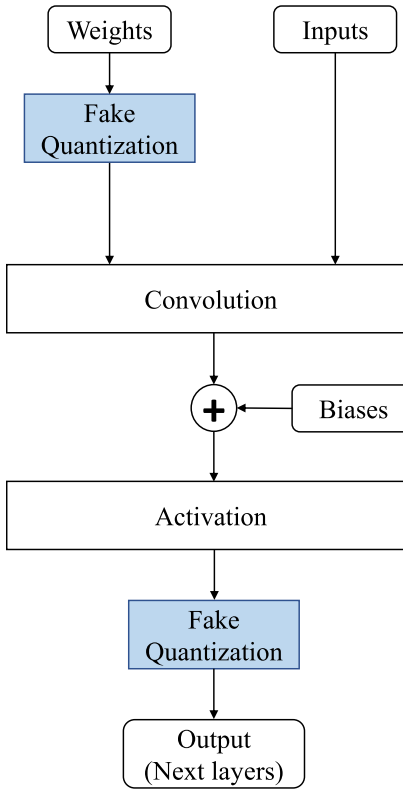


FIGURE 6. Illustration of a typical CNN layer integrated with fake quantization modules during QAT. This approach facilitates a smooth transition to low-bit inference deployment while minimizing loss in classification accuracy.

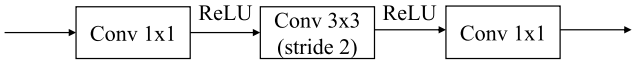


FIGURE 7. Bottleneck-like layer for temporary channel reduction. The first 1 × 1 convolution reduces the channel dimensions for the 3 × 3 convolution, and finally restores the dimensions.

QAT workflow
1: Replace original layers with QAT layers (inject fake quantization, observers).
2: Train the QAT model.
3: Replace all the QAT layers with the original layers.
4: Convert the model into a quantized int8 network.

F. CNN MODELS AND TRAINING ALGORITHM

In computer vision applications, including object detection, semantic segmentation, and fault diagnosis, CNNs have garnered significant attention owing to their high accuracy and ability to automatically learn discriminative features from raw or transformed signals. CNN architectures for bearing fault diagnosis can range from lightweight custom models to well-known backbones such as AlexNet or ResNet.

ResNet-50 is a deep CNN architecture that belongs to the ResNet family, and was introduced by Kaiming et al. [52]. The ResNet framework is a breakthrough in addressing the degradation problem that arises when very deep neural networks are trained, and the phenomenon in which adding more layers can lead to higher training errors rather than improved accuracy. By introducing residual (skip) connections, ResNet-50 can learn more efficiently even with a substantial number of layers. This innovation ultimately opened the door to the construction of neural networks with hundreds or even thousands of layers, without suffering from vanishing or exploding gradients to the same degree as prior architectures. One notable feature of ResNet-50 is its use as a bottleneck. Instead of repeatedly applying 3×3 convolutions, each block typically employs a $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ stack (Fig. 7). This strategy reduces the computational cost and number of parameters compared with the naïve approach of stacking multiple 3×3 convolutions.

Despite its high accuracy and ease of training in classification tasks, the ResNet-50 architecture is relatively large and computationally intensive, which makes its deployment on mobile or embedded devices with limited resources challenging. MobileNet, introduced by Howard at Google [53], addresses this gap by tailoring the CNN architecture specifically for resource-constrained environments and striking a balance between the model size, speed, and accuracy. Hence, in this study, we selected MobileNetV2 and MobileNetV3-Large as lightweight CNN architectures for the student models.

For the training algorithm in the offline phase, we used an Adam optimizer with a learning rate of 0.001 (stages 1 and 2) and 0.0001 (stage 3) and a batch size of 32. The criterion used to calculate the loss function was cross-entropy. We also employed a learning rate scheduler that adjusts the learning rate over time to balance the rapid initial learning with stable convergence. The scheduler monitors the validation loss and when improvements in this metric stagnate, it automatically reduces the learning rate, allowing the model to further refine its weights.

IV. EXPERIMENTS

This section describes the experimental design and methodological approach used to evaluate and validate the effectiveness of the proposed method. The results obtained under diverse conditions were compared and evaluated to demonstrate process performance across a wide range of scenarios.

A. EXPERIMENTAL SETUP

1) TESTBED AND DATA-ACQUISITION SYSTEM

An experimental apparatus that meets the requirements for bearing fault diagnosis has been extensively utilized in previous investigations [54], [55], comprising two shafts (one at the driven end and one at the non-driven end) coupled through a gearbox with a damping ratio of 1.52:1 (Fig. 8). Each shaft was mounted on a testbed and attached to a cylindrical bearing (FAG NJ206-E-TVP2). As illustrated in Fig. 9, the non-driven-end shaft is linked to an adjustable blade via a pulley, providing a controllable load for the system. A speed controller can be used to change the rotational speed of the 3-phase induction motor. To prevent shock transmission, we connected the motor and driven-end shafts using a flexible coupling.

A wideband AE sensor was affixed to a non-drive-end bearing to acquire high-frequency signals. AE data were collected using a PCI-2-based acquisition board operated at a sampling rate of 250 kHz. The AE sensor (PAC WSα features an effective frequency range of 100–900 kHz, $\pm$1.5 dB directionally, -62 dB peak sensitivity, and a resonant frequency at

TABLE 2. Detailed specifications of the test bearing used in the experimental setup.

Pitch diameter	46.5 (mm)
Rolling element diameter	9.0 (mm)
Contact angle	0°
Number of rolling elements	13

TABLE 3. Dataset for the proposed method. Each subset contains 8 classes (7 fault cases and 1 normal case) with uniform distribution for balanced learning. The dataset covers three crack sizes and six rotational speeds to enhance generalization across various operating conditions.

Subset	Crack size			Speed (RPM)	Samples
	Length (mm)	Width (mm)	Depth (mm)		
Training	3	0.6	0.3	300, 400, 500	4800
	6	0.6	0.5		
	12	0.6	0.5		
Validation	3	0.6	0.3	250, 350, 450	1200
	6	0.6	0.5		
	12	0.6	0.5		
Testing	3	0.6	0.3	250, 350, 450	2400
	6	0.6	0.5		
	12	0.6	0.5		

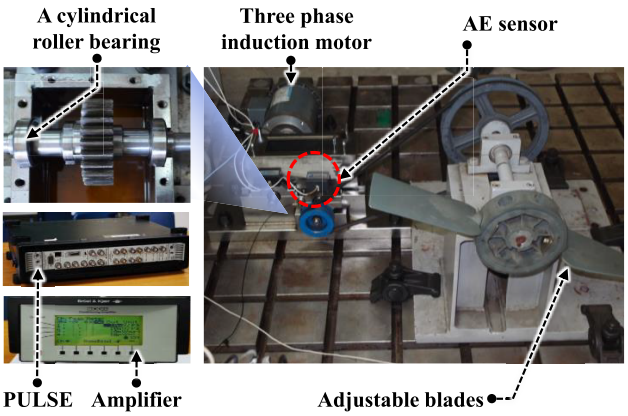


FIGURE 8. Experimental testbed and data acquisition system. This configuration facilitates precise monitoring and analysis of bearing faults under controlled mechanical and operational conditions.

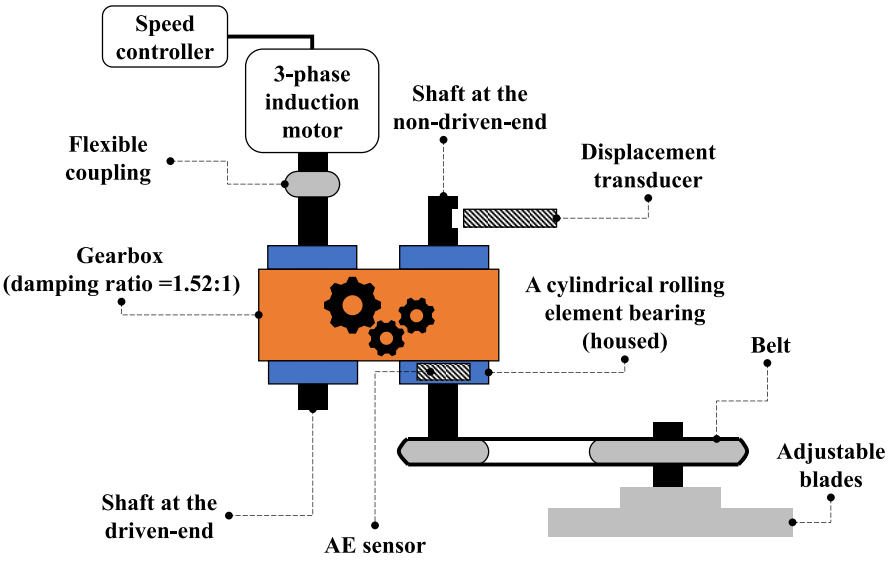


FIGURE 9. Diagram of the experimental testbed. Key components include a motor-gearbox assembly and instruments (AE sensor and displacement), which enable signal acquisition under diverse operating conditions.

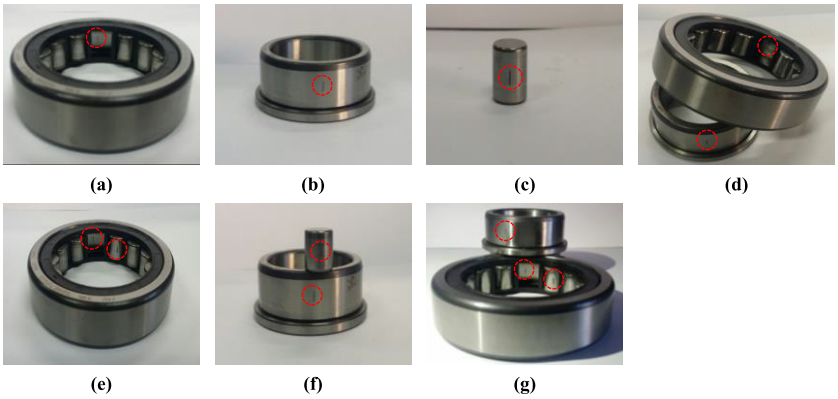


FIGURE 10. Single and compound bearing faults: (a) outer fault (BCO), (b) inner fault (BCI), (c) roller fault (BCR), (d) inner and outer fault (BCIO), (e) outer and roller fault (BCOR), (f) inner and roller fault (BCIR), (g) inner, outer, and roller fault (BCIOR).

650 kHz. The specifications of the experimental bearings are listed in Table 2.

2) DESKTOP SYSTEM

The experiments were conducted on a high-performance workstation equipped with an AMD Ryzen 9 5950X 16-Core CPU, 128 GB RAM, and an NVIDIA GeForce RTX 3090 GPU. This setup provides a balanced and robust environment for both computationally heavy tasks and real-time monitoring of the experimental results.

MATLAB and PyTorch were used for the different workflow stages. MATLAB, which is known for its extensive library of signal-processing toolboxes and user-friendly interactive environments, was used to preprocess the raw signals and generate spectrograms. Subsequently, PyTorch was used to implement and train the deep learning models. The synergy between MATLAB-based signal handling and PyTorch-based model training facilitated a streamlined pipeline, allowing iterative improvements in data transformation and the rapid prototyping of neural network architectures.

3) EMBEDDED SYSTEM

For edge deployment, a Raspberry Pi 3 Model B was employed as the embedded system, offering a compact, energy-efficient hardware platform. The Raspberry Pi 3 B was powered by a quad-core ARM Cortex-A53 CPU running at 1.2 GHz and containing 1 GB of LPDDR2 RAM. Although modest compared to modern desktop-class hardware, this board can still execute lightweight deep learning models or optimized inference routines, provided that the models are tuned for memory and computational efficiency.

TABLE 4. Processing time of different time–frequency analysis techniques measured in MATLAB. The results demonstrate the superior efficiency of STFT for fast signal preprocessing.

	Average processing time of 1 signal (ms)	Average processing time of 1 segmented signal (ms)
STFT	19.7	1.2
CQT	164.6	13.9
HHT	573.4	27.7

TABLE 5. Comparison between STFT and grayscale spectrograms in terms of storage requirements and classification accuracy. Grayscale spectrograms enable more efficient deployment without compromising diagnostic performance.

	Average size of 1 image (KB)	Accuracy (%)
STFT Spectrogram	288	99.5
Grayscale Spectrogram	80	99.88

TABLE 6. Number of MAC operations for different input tensor sizes (channel depth, width × height). The results indicate that smaller tensor sizes and fusion strategies contribute to reduced computational cost.

	(3, 224×224)	(3, 64×64)	(3, 32×32)	(1, 224×224)	(1, 64×64)	(1, 32×32)
Number of MAC operations (KMac)	4130000	337190	84310	4050000	330770	82700
Number of MAC operations after layer fusion (KMac)	4107750	335375	83856	4027750	328955	82246

A Linux-based operating system, typically a Raspberry Pi OS (formerly Raspbian), was installed to provide a stable foundation for running inference workloads on the boards. In addition to this OS, PyTorch was used to execute the deep learning inference.

B. EXPERIMENTAL METHODS AND RESULTS

1) DATASET

Using diamond cutting, we introduced multiple crack sizes into the bearings with distinct fault types at different positions (Fig. 10). The resulting dataset included three crack dimensions (3 mm, 6 mm, and 12 mm) across seven single- or compound-bearing faults and one normal condition, each tested at six distinct rotational speeds. All data samples were obtained by segmenting the AE signals into 0.05 s intervals, and the complete dataset was then partitioned into training, validation, and testing subsets, as summarized in Table 3. The training and validation subsets inform the model-development process, whereas the testing subset is reserved for evaluating the overall performance of the proposed method.

2) PERFORMANCE OF THE PROPOSED METHOD

Each sample in these subsets underwent an STFT transformation to generate a spectral representation that was subsequently converted into a grayscale spectrogram. To evaluate the computational efficiency of our signal-processing method, we benchmarked the processing time required by the STFT, CQT, and HHT on full and segmented AE signals using MATLAB on our desktop system. As shown in Table 4, STFT demonstrated the highest efficiency, requiring only 19.7 ms to process a full signal and just 1.2 ms per segmented signal. This performance makes STFT highly suitable for resource-constrained environments where real-time fault diagnosis is essential. The CQT, although offering improved frequency resolution in lower frequency bands, incurred a higher computational cost, 164.6 ms for the full signal and 13.9 ms for one segment. The HHT exhibited the highest processing time owing to the iterative nature of EMD. These results underscore the trade-offs between representational power and computational cost, highlighting STFT as the most practical technique for real-time edge deployment and suggesting that CQT and HHT are more appropriate for offline analysis or latency-tolerant applications.

The transformed single-channel grayscale spectrograms achieved a significant storage reduction, occupying only one-third of the storage space required by 3-channel STFT spectrograms. This demonstrates the efficiency of grayscale spectrograms for bearing fault diagnosis in resource-limited environments. In our experiments using the ResNet-50 model, as shown in Table 5, the grayscale spectrogram

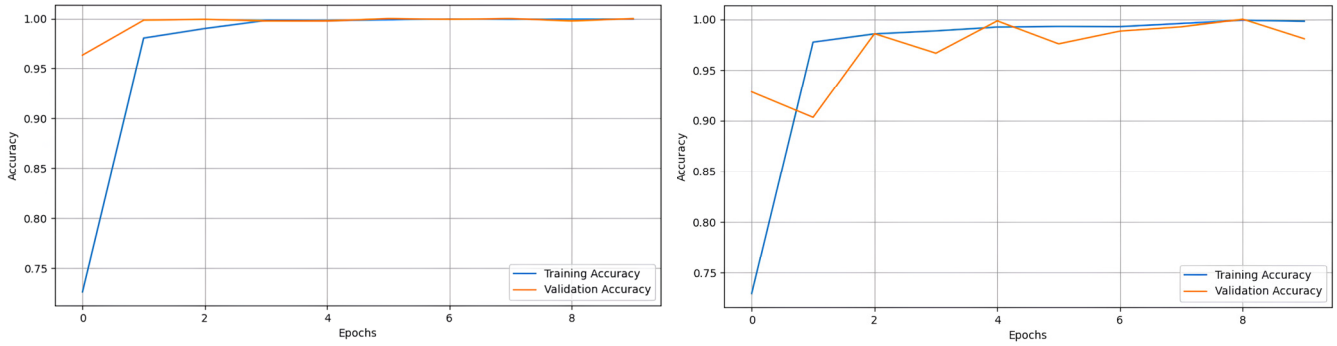


FIGURE 11. Performance of MobileNetV2 with KD (left) and non-KD (right). The KD model shows stable validation behavior, whereas the non-KD model exhibits larger fluctuations in the validation curve during early epochs.

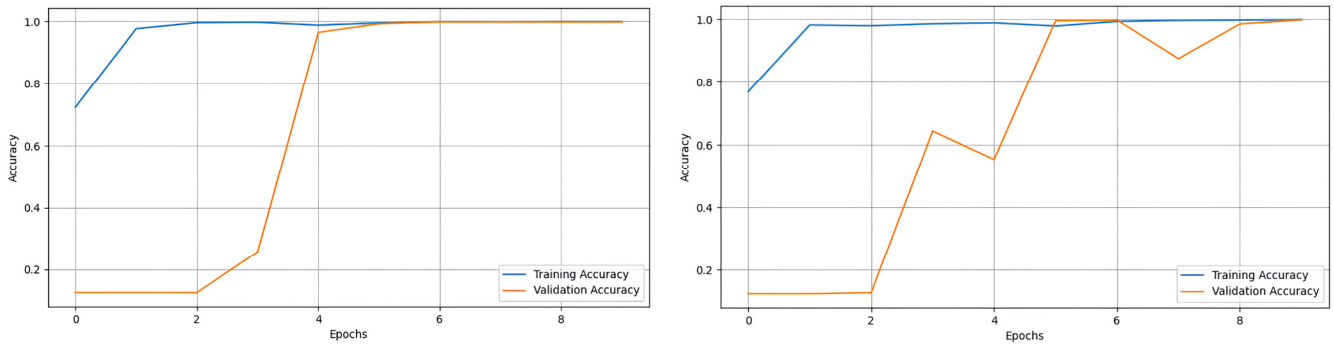


FIGURE 12. Performance of MobileNetV3-L with KD (left) and non-KD (right). The KD model shows stable validation behavior, whereas the non-KD model exhibits larger fluctuations in the validation curve during early epochs.

TABLE 7. Comprehensive assessment of the proposed method under varying input resolutions and model architectures.

Input size	Model	Memory footprint (MB)	Accuracy (%)	Inference latency (AMD Ryzen 9 5950X) (ms/sample)
224×224	ResNet-50	99.03	99.83	30.21
	MobileNetV2	8.76	99.92	12.61
	Quantized MobileNetV2	2.52	99.90	6.17
	MobileNetV3-L	16.27	99.79	12.04
	Quantized MobileNetV3-L	4.48	99.80	10.06
64×64	ResNet-50	99.03	99.83	14.85
	MobileNetV2	8.76	98.71	5.80
	Quantized MobileNetV2	2.52	99.80	2.96
	MobileNetV3-L	16.27	99.46	6.00
	Quantized MobileNetV3-L	4.48	99.70	4.83
32×32	ResNet-50	99.03	99.00	9.84
	MobileNetV2	8.76	98.13	4.58
	Quantized MobileNetV2	2.52	98.60	2.30
	MobileNetV3-L	16.27	99.13	6.58
	Quantized MobileNetV3-L	4.48	98.90	3.57

representation achieved nearly identical classification accuracy as the original STFT spectrograms. Despite aggressive compression, the preservation of diagnostic performance suggests that essential discriminative features are retained in the grayscale images. Moreover, the one-channel representation simplifies the input for CNN-based models, lowering the number of learnable parameters in the first convolutional layer and accelerating the inference. Table 6 highlights the impact of channel depth on the overall number of MAC operations. For identical spatial dimensions, grayscale images consistently required fewer MACs than their STFT counterparts, reflecting a 1.9% reduction in computational cost. This difference can be attributed directly to the reduced number of input channels processed by the first convolutional layer.

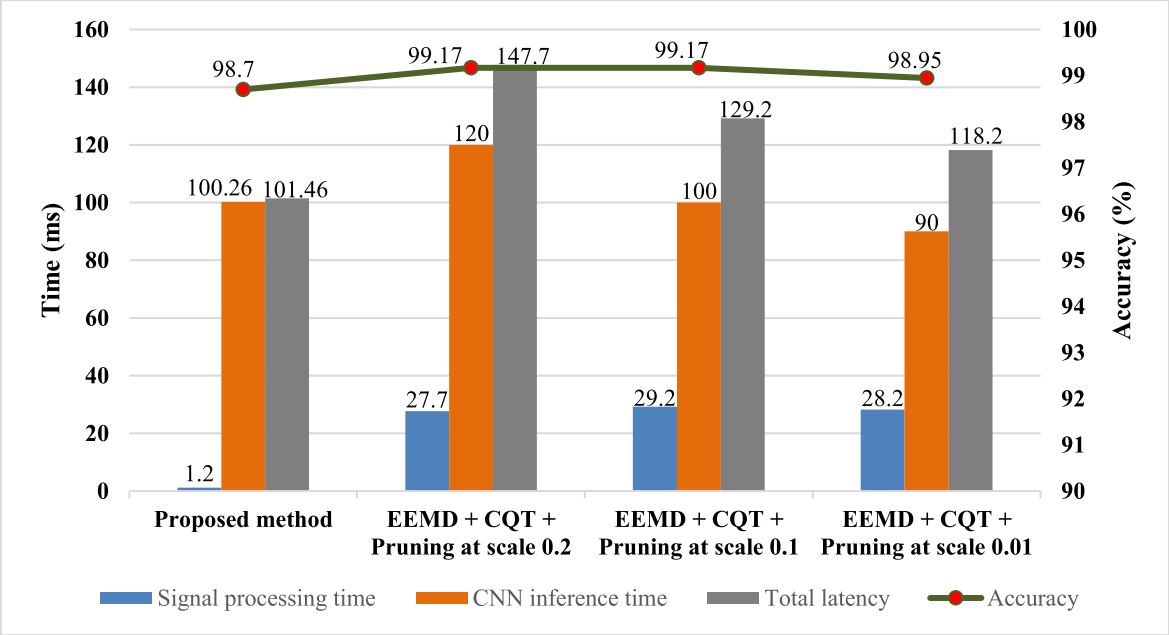


FIGURE 13. Comparison of latency and accuracy of MobileNetV2 between the proposed method and other strategies on the Raspberry Pi 3 Model B.

Table 6 also shows that decreasing the input resolution yielded a substantial reduction in the number of MAC operations. For 3-channel RGB inputs, reducing the resolution from 224 × 224 to 64 × 64 decreased the number of MACs from 4,130,000 KMac to 337,190 KMac, an approximate reduction of 91.8%. Further reduction to 32 × 32 yielded a total drop of nearly 98% compared with the full-resolution input. The same trend was observed for the 1-channel grayscale spectrograms, where MACs declined from 4,050,000 KMac at 224 × 224 to 330,770 KMac at 64 × 64, and further down to 82,700 KMac at 32 × 32. These consistent reductions across both RGB and grayscale formats confirm the direct relationship between the input resolution and inference cost.

Subsequently, to reduce the model size and number of parameters, we used KD to guide the training of our much smaller student networks, the MobileNetV2 and MobileNetV3-Large architectures, for comparison purposes. KD-based approaches appear to offer a more stable and efficient training process and demonstrate notable differences in convergence behavior and generalization performance compared with non-KD conditions. By leveraging the knowledge of the teacher model, student models avoid the need to explore suboptimal solutions during early training epochs. This stability is evident in the absence of large fluctuations in the validation accuracy, as opposed to the non-KD cases, where the validation curves displayed inconsistent performance in earlier epochs (Figs. 11 and 12).

To further minimize the computational overhead during inference, layer fusion was adopted to streamline consecutive operations such as convolutions, BN, and activation functions into a single, more efficient computational block. As shown in Table 6, this consistently reduced the number of MAC operations across all evaluated input configurations, independent of the resolution or input channel depth. This demonstrates that the layer fusion contributes to predictable and systematic gains by eliminating redundant intermediate operations and reducing the memory access overhead.

We applied QAT to each fused model to enable the efficient deployment of edge devices. Table 7 illustrates the effect of model quantization by comparing the performance of MobileNetV2 and MobileNetV3-L before and after quantization across three input resolutions: 224 × 224, 64 × 64, and 32 × 32. The results demonstrate that QAT consistently reduces both the memory footprint and inference latency while maintaining high classification accuracy across all input sizes. The memory footprint of the model did not vary with changes in the input image resolution. For MobileNetV2, quantization provides a substantial memory reduction from 8.76 MB to 2.52 MB, representing a 71% decrease across all resolutions. Similarly, MobileNetV3-L showed a reduction from 16.27 MB to 4.48 MB, which is approximately 72% less than the memory usage. These improvements are particularly beneficial for memory-constrained embedded systems. In terms of inference latency, the quantized models achieved faster computations for all input sizes. For example, at a 32 × 32 resolution, the latency of MobileNetV2 drops from 4.58 ms/sample to 2.30 ms/sample after quantization, and for MobileNetV3-L, it decreases from 6.58 ms/sample to 3.57 ms/sample, representing nearly a 50% latency reduction. These gains enhance

the real-time applicability of these models for edge AI tasks. Notably, these improvements were obtained without compromising the accuracy. MobileNetV2's accuracy slightly improved from 98.13% to 98.60%, whereas MobileNetV3-L maintained a high accuracy with only a minor decrease from 99.13% to 98.90%. Overall, the results confirm that QAT is a highly effective technique for improving the computational efficiency and memory scalability, especially when combined with compact models such as MobileNetV2 and V3. The results in Table 7 show that MobileNetV2 achieved the best performance at 32 × 32 resolution in terms of efficiency.

3) EDGE DEPLOYMENT

To validate the practical efficiency of the proposed method, the inference latency of the final MobileNetV2 (input resolution: 32 × 32 pixels) deployed on an embedded platform was evaluated. We compared our method with pruning-based techniques from [15], which reduced model complexity by removing redundant parameters at various scaling levels (0.2, 0.1, and 0.01). As shown in Fig. 13, our method achieved a CNN inference latency of 100.26 ms, with an extremely lightweight signal-processing time of just 1.2 ms, resulting in a total latency of 101.46 ms. This outperforms all pruning-based models, including the most aggressively compressed model at scale 0.01, which still provides a higher total latency of 118.2 ms due to its costly signal-processing stage. Whereas the pruned models in [15] employed the EEMD and CQT frameworks, which demand up to 29.2 ms for signal preprocessing, our method benefits from a much more efficient STFT-based transformation and grayscale image representation. Moreover, our method achieves a competitive accuracy of 98.7%, compared to 99.17% at scale 0.2 and 98.95% at scale 0.01, as reported in [15]. These findings clearly demonstrate that our approach offers a superior balance between inference speed, accuracy, and computational efficiency, making it suitable for embedded and edge-based fault diagnosis systems, where processing power and latency are tightly constrained.

V. CONCLUSION

In this study, we propose a novel approach that utilizes the computationally efficient processing of AE signals combined with a hybrid neural network compression technique to demonstrate the feasibility of deploying bearing fault diagnosis across variable rotational speeds in embedded systems. The proposed method achieved inference times and memory footprints suitable for highly resource-constrained environments such as Raspberry Pi board platforms, while preserving high diagnostic accuracy. Nevertheless, this study highlights several points that merit further investigation in future research. First, the dataset used in this study was collected under controlled, idealized conditions and may not fully reflect the challenges encountered in real-world industrial environments, where nonstationary operating conditions often lead to significant variations in data distributions. Second, the data preprocessing and inference processes were conducted independently rather than as an integrated pipeline, potentially limiting the evaluation of the effectiveness of the method under practical deployment scenarios. In addition, the power consumption remains a critical factor when implementing deep learning models on edge devices, particularly in energy-constrained environments. In future work, we plan to explore energy-efficient approaches that will enable the deployment of our models to ultra-resource-constrained microcontrollers. Specifically, we intend to target low-bit platforms, such as 16-bit or 32-bit CPUs, operating under battery-powered conditions. Through these efforts, we hope to advance the practical applicability and scalability of our approach in real-world industrial environments.

REFERENCES

[1] S. A. Khan and J.-M. Kim, "Automated bearing fault diagnosis using 2D analysis of vibration acceleration signals under variable speed conditions," *Shock Vibrat.*, vol. 2016, pp. 1–11, Nov. 2016, doi: 10.1155/2016/8729572.

[2] L. Cui, X. Wang, H. Wang, and J. Ma, "Research on remaining useful life prediction of rolling element bearings based on time-varying Kalman filter," *IEEE Trans. Instrum. Meas.*, vol. 69, no. 6, pp. 2858–2867, Jun. 2020, doi: 10.1109/TIM.2019.2924509.

[3] M. T. Pham, J.-M. Kim, and C. H. Kim, "Accurate bearing fault diagnosis under variable shaft speed using convolutional neural networks and vibration spectrogram," *Appl. Sci.*, vol. 10, no. 18, p. 6385, Sep. 2020, doi: 10.3390/app10186385.

[4] B. Dolenc, P. Boškoski, and D. Juričić, "Distributed bearing fault diagnosis based on vibration analysis," *Mech. Syst. Signal Process.*, vols. 66–67, pp. 521–532, Jan. 2016, doi: 10.1016/j.ymssp.2015.06.007.

[5] D. Li, M. Li, L. Yang, X. Wang, F. Zhang, and Y. Liang, "Rolling bearing fault diagnosis in strong noise background based on vibration signals," *Signal, Image Video Process.*, vol. 18, no. 2, pp. 1295–1303, Mar. 2024, doi: 10.1007/s11760-023-02846-y.

[6] R. Wells, M. A. Hamstad, and A. K. Mukherjee, "On the origin of the first peak of acoustic emission in 7075 aluminium alloy," *J. Mater. Sci.*, vol. 18, no. 4, pp. 1015–1020, Apr. 1983, doi: 10.1007/bf00551968.

[7] L. Tang, X. Wu, D. Wang, and X. Liu, "A comparative experimental study of vibration and acoustic emission on fault diagnosis of low-speed bearing," *IEEE Trans. Instrum. Meas.*, vol. 72, 2023, Art. no. 3529211, doi: 10.1109/TIM.2023.3312761.

[8] T. A. Nguyen and J. Lee, "A nonlinear convolutional neural network-based equalizer for holographic data storage systems," *Appl. Sci.*, vol. 13, no. 24, p. 13029, Dec. 2023, doi: 10.3390/app132413029.

[9] V.-P. Bui, T. Q. Dinh, I. Leyva-Mayorga, S. R. Pandey, E. Lagunas, and P. Popovski, "On-board change detection for resource-efficient Earth observation with LEO satellites," in *Proc. IEEE Global Commun. Conf.*, Kuala Lumpur, Malaysia, Dec. 2023, pp. 3971–3976, doi: 10.1109/GLOBECOM54140.2023.10437543.

[10] Y. Tang, C. Zhang, J. Wu, Y. Xie, W. Shen, and J. Wu, "Deep learning-based bearing fault diagnosis using a trusted multiscale quadratic attention-embedded convolutional neural network," *IEEE Trans. Instrum. Meas.*, vol. 73, 2024, Art. no. 3513215, doi: 10.1109/TIM.2024.3374311.

[11] S. Zhang, S. Zhang, B. Wang, and T. G. Habetler, "Deep learning algorithms for bearing fault diagnostics—A comprehensive review," *IEEE Access*, vol. 8, pp. 29857–29881, 2020, doi: 10.1109/ACCESS.2020.2972859.

[12] S. Anbalagan, B. Natarajan, and B. Srinivasan, "A physics based deep learning approach for cross domain bearing fault detection," in *Proc. IEEE Kansas Power Energy Conf. (KPEC)*, Manhattan, KS, USA, Apr. 2023, pp. 1–4, doi: 10.1109/KPEC58008.2023.10215413.

[13] L. Wan, Y. Chen, H. Li, and C. Li, "Rolling-element bearing fault diagnosis using improved LeNet-5 network," *Sensors*, vol. 20, no. 6, p. 1693, Mar. 2020, doi: 10.3390/s20061693.

[14] X. Chen, J. Li, and Y. Zhao, "Hardware resource and computational density efficient CNN accelerator design based on FPGA," in *Proc. IEEE Int. Conf. Integr. Circuits, Technol. Appl. (ICTA)*, Zhuhai, China, Nov. 2021, pp. 204–205, doi: 10.1109/ICTA53157.2021.9661886.

[15] M. T. Pham, J.-M. Kim, and C. H. Kim, "Deep learning-based bearing fault diagnosis method for embedded systems," *Sensors*, vol. 20, no. 23, p. 6886, Dec. 2020, doi: 10.3390/s20236886.

[16] M. T. Pham, J.-M. Kim, and C. H. Kim, "Efficient fault diagnosis of rolling bearings using neural network architecture search and sharing weights," *IEEE Access*, vol. 9, pp. 98800–98811, 2021, doi: 10.1109/ACCESS.2021.3096036.

[17] J. Yang, L. Feng, and Y. Lu, "Rolling bearing fault diagnosis method based on enhanced knowledge distillation and lightweight convolution network," in *Proc. 4th Int. Conf. Comput. Eng. Intell. Control (ICCEIC)*, Guangzhou, China, Oct. 2023, pp. 620–627, doi: 10.1109/icceic60201.2023.10426666.

[18] A. Chehade, E. Ragusa, P. Gastaldo, and R. Zunino, "Energy-efficient deep learning for traffic classification on microcontrollers," in *Proc. IEEE ISCC*, Jun. 2025, pp. 1–12.

[19] Z.-S. Syu and C.-H. Lee, "One-dimensional binary convolutional neural network accelerator design for bearing fault diagnosis," *IEEE Sensors J.*, vol. 24, no. 3, pp. 3649–3658, Feb. 2024, doi: 10.1109/JSEN.2023.3340715.

[20] K.-C.-J. Chen and W.-R. Syu, "High reliable and accurate stochastic computing-based artificial neural network architecture design," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Singapore, May 2024, pp. 1–5, doi: 10.1109/ISCAS58744.2024.10558634.

[21] C. F. Frasser, P. Linares-Serrano, I. D. D. L. Ríos, A. Morán, E. S. Skibinsky-Gitlin, J. Font-Rosselló, V. Canals, M. Roca, T. Serrano-Gotarredona, and J. L. Rosselló, "Fully parallel stochastic computing hardware implementation of convolutional neural networks for edge computing applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 12, pp. 10408–10418, Dec. 2023, doi: 10.1109/TNNLS.2022.3166799.

[22] S. E. Maachi, A. Chehri, and R. Saadane, "Efficient hardware acceleration of spiking neural networks using FPGA: Towards real-time edge neuromorphic computing," in *Proc. IEEE 99th Veh. Technol. Conf. (VTC-Spring)*, Singapore, Jun. 2024, pp. 1–5, doi: 10.1109/VTC2024-SPRING62846.2024.10683049.

[23] G. Shen, D. Zhao, and Y. Zeng, "Exploiting high performance spiking neural networks with efficient spiking patterns," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 9, no. 2, pp. 1480–1489, Apr. 2025, doi: 10.1109/TETCI.2025.3540408.

[24] E. Ragusa, F. Zonzini, L. De Marchi, and R. Zunino, "Compression–accuracy co-optimization through hardware-aware neural architecture search for vibration damage detection," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31745–31757, Oct. 2024, doi: 10.1109/JIOT.2024.3419251.

[25] E. Ragusa, F. Zonzini, P. Gastaldo, and L. De Marchi, "Combining compressed sensing and neural architecture search for sensor-near vibration diagnostics," *IEEE Trans. Ind. Informat.*, vol. 20, no. 8, pp. 10488–10498, Aug. 2024, doi: 10.1109/TII.2024.3395648.

[26] A. M. Garavagno, E. Ragusa, A. Frisoli, and P. Gastaldo, "An affordable hardware-aware neural architecture search for deploying convolutional neural networks on ultra-low-power computing platforms," *IEEE Sensors Lett.*, vol. 8, no. 5, pp. 1–4, May 2024, doi: 10.1109/LSENS.2024.3387056.

[27] O. T.-C. Chen, Y.-X. Chang, C.-Y. Chung, Y.-Y. Cheng, and H. A. Manh-Hung, "Hardware-aware iterative one-shot neural architecture search with adaptable knowledge distillation for efficient edge computing," *IEEE Access*, vol. 13, pp. 54204–54222, 2025, doi: 10.1109/ACCESS.2025.3554185.

[28] Y. Hou, J. Wang, Z. Chen, J. Ma, and T. Li, "Diagnosisformer: An efficient rolling bearing fault diagnosis method based on improved transformer," *Eng. Appl. Artif. Intell.*, vol. 124, Sep. 2023, Art. no. 106507, doi: 10.1016/j.engappai.2023.106507.

[29] X. Wang, T. Ying, and W. Tian, "Spectrum representation based on STFT," in *Proc. 13th Int. Congr. Image Signal Process., Biomed. Eng. Informat. (CISP-BMEI)*, Chengdu, China, Oct. 2020, pp. 435–438, doi: 10.1109/CISP-BMEI51763.2020.9263516.

[30] H. Cheng, S. Yu, and L. Cheng, "Application of wavelet transform in fault diagnosis of rolling bearing," in *Proc. 10th Int. Conf. Natural Comput. (ICNC)*, Xiamen, China, Aug. 2014, pp. 1066–1070, doi: 10.1109/ICNC.2014.6975988.

[31] W. Xi, L. Bai, M. Hui, and Q. Wu, "A novel rolling bearing fault detect method based on empirical wavelet transform," in *Proc. 13th IEEE Conf. Ind. Electron. Appl. (ICIEA)*, Wuhan, China, May 2018, pp. 2764–2768, doi: 10.1109/ICIEA.2018.8398179.

[32] D. Du, J. Zhang, Y. Fang, and J. Tian, "Motor bearing fault diagnosis based on Hilbert–Huang transform and convolutional neural networks," in *Proc. IEEE Transp. Electrific. Conf. Expo, Asia–Pacific (ITEC Asia–Pacific)*, Haining, China, Oct. 2022, pp. 1–5, doi: 10.1109/ITECASIA-PACIFIC56316.2022.9941910.

[33] N. D. Thuan, N. Thi Hue, P. Q. Vuong, and H. S. Hong, "Intelligent bearing fault diagnosis with a lightweight neural network," in *Proc. 11th Int. Conf. Control, Autom. Inf. Sci. (ICCAIS)*, Hanoi, Vietnam, Nov. 2022, pp. 261–266, doi: 10.1109/ICCAIS56082.2022.9990211.

[34] A. Choudhary, T. Mian, S. Fatima, and B. K. Panigrahi, "Deep transfer learning based fault diagnosis of electric vehicle motor," in *Proc. IEEE Int. Conf. Power Electron., Drives Energy Syst. (PEDES)*, Jaipur, India, Dec. 2022, pp. 1–6, doi: 10.1109/PEDES56012.2022.10080274.

[35] C. Mateo and J. A. Talavera, "Bridging the gap between the short-time Fourier transform (STFT), wavelets, the constant-Q transform and multi-resolution STFT," *Signal, Image Video Process.*, vol. 14, no. 8, pp. 1535–1543, Nov. 2020, doi: 10.1007/s11760-020-01701-8.

[36] C. Lian, Y. Zhao, T. Sun, F. Dong, Z. Zhan, and L. Xin, "A new time series data imaging scheme for mechanical fault diagnosis," *IEEE Trans. Instrum. Meas.*, vol. 73, 2024, Art. no. 3512711, doi: 10.1109/TIM.2024.3370796.

[37] C. Lian, Y. Zhao, J. Shao, T. Sun, F. Dong, Z. Ju, Z. Zhan, and P. Shan, "CFI-LFENet: Infusing cross-domain fusion image and lightweight feature enhanced network for fault diagnosis," *Inf. Fusion*, vol. 104, Apr. 2024, Art. no. 102162, doi: 10.1016/j.inffus.2023.102162.

[38] J. Yu, J. Huang, X. Fu, Y. Jin, X. Yan, and X. Yang, "Disentangled feature representation based on multiscale content learning in industrial heterogeneous nonstationary fault detection," *IEEE Trans. Instrum. Meas.*, vol. 74, pp. 1–12, 2025, doi: 10.1109/TIM.2025.3551823.

[39] J. Yu, Z. Lv, H. Ruan, S. Hu, Q. Jiang, X. Yan, Y. Liu, and X. Yang, "Mutual stacked autoencoder for unsupervised fault detection under complex multi-residual correlations," *Adv. Eng. Informat.*, vol. 62, Oct. 2024, Art. no. 102837, doi: 10.1016/j.aei.2024.102837.

[40] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," in *Proc. 28th Int. Conf. Neural Inf. Process. Syst.*, Montreal, QC, Canada, 2015, pp. 1135–1143, doi: 10.5555/2969239.2969366.

[41] S. Liu, Q. Li, B. Peng, Y. Qiao, M. Lin, and Q. Zeng, "An energy-constrained optimization-based structured pruning method for deep neural network compression," in *Proc. IEEE 22nd Int. Conf. Commun. Technol. (ICCT)*, Nanjing, China, Nov. 2022, pp. 1903–1907, doi: 10.1109/ICCT56141.2022.10073392.

[42] S. Wang, P. Lin, R. Hu, H. Wang, J. He, Q. Huang, and S. Chang, "Acceleration of LSTM with structured pruning method on FPGA," *IEEE Access*, vol. 7, pp. 62930–62937, 2019, doi: 10.1109/ACCESS.2019.2917312.

[43] A. Ding, Y. Qin, B. Wang, L. Jia, and X. Cheng, "Lightweight multiscale convolutional networks with adaptive pruning for intelligent fault diagnosis of train bogie bearings in edge computing scenarios," *IEEE Trans. Instrum. Meas.*, vol. 72, 2023, Art. no. 3502813, doi: 10.1109/TIM.2022.3231325.

[44] T. Simons and D.-J. Lee, "A review of binarized neural networks," *Electronics*, vol. 8, no. 6, p. 661, Jun. 2019, doi: 10.3390/electronics8060661.

[45] B. Liu, F. Li, X. Wang, B. Zhang, and J. Yan, "Ternary weight networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Rhodes Island, Greece, Jun. 2023, pp. 1–5, doi: 10.1109/ICASSP49357.2023.10094626.

[46] X. Li, J. Li, and Z. Lv, "ShuffleNet2MC: A method of light weight fault diagnosis," in *Proc. Int. Conf. Comput. Eng. Appl. (ICCEA)*, Kunming, China, Jun. 2021, pp. 264–270, doi: 10.1109/ICCEA53728.2021.00060.

[47] C.-C. Chung, W.-T. Chen, and Y.-C. Chang, "Using quantization-aware training technique with post-training fine-tuning quantization to implement a MobileNet hardware accelerator," in *Proc. Indo-Taiwan 2nd Int. Conf. Comput., Anal. Netw.*, Rajpura, India, Feb. 2020, pp. 28–32, doi: 10.1109/Indo-TaiwanICAN48429.2020.9181327.
[48] J. Ma, W. Cai, Y. Shan, Y. Xia, and R. Zhang, "An integrated framework for bearing fault diagnosis: Convolutional neural network model compression through knowledge distillation," *IEEE Sensors J.*, vol. 24, no. 23, pp. 40083–40095, Dec. 2024, doi: 10.1109/JSEN.2024.3481298.
[49] M. T. Pham, J.-M. Kim, and C. H. Kim, "Rolling bearing fault diagnosis based on improved GAN and 2-D representation of acoustic emission signals," *IEEE Access*, vol. 10, pp. 78056–78069, 2022, doi: 10.1109/ACCESS.2022.3193244.
[50] H. Zhivomirov, "On the development of STFT-analysis and ISTFT-synthesis routines and their practical implementation," *TEM J.*, vol. 8, pp. 56–64, Feb. 2019, doi: 10.18421/tem81-07.
[51] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
[52] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.
[53] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.
[54] M. Kang, J. Kim, I.-K. Jeong, J.-M. Kim, and M. Pecht, "A massively parallel approach to real-time bearing fault detection using sub-band analysis on an FPGA-based multicore system," *IEEE Trans. Ind. Electron.*, vol. 63, no. 10, pp. 6325–6335, Oct. 2016, doi: 10.1109/TIE.2016.2574986.
[55] H. N. Nguyen, C.-H. Kim, and J.-M. Kim, "Effective prediction of bearing fault degradation under different crack sizes using a deep neural network," *Appl. Sci.*, vol. 8, no. 11, p. 2332, Nov. 2018, doi: 10.3390/app8112332.



HUYNH-ANH-HUY NGUYEN received the B.E. degree in control engineering and automation from Ho Chi Minh City University of Technology, Ho Chi Minh City, Vietnam, in 2019. He is currently pursuing the M.S. degree with the Graduate School of Computer Science and Engineering, Soongsil University, Seoul, South Korea. From 2019 to 2023, he was an Embedded Software Engineer with Viettel High Tech, Vietnam. He was also a Senior Embedded Software Engineer with Bosch Global Software Technologies, Vietnam, from 2023 to 2024. His research interests include computer architecture, embedded systems, signal processing, deep learning, and fault diagnosis.



CHEOL HONG KIM received the B.S. degree in computer engineering from Seoul National University, Seoul, South Korea, in 1998, the M.S. degree, in 2000, and the Ph.D. degree in electrical and computer engineering from Seoul National University, in 2006. He was a Senior Engineer with Samsung Electronics, South Korea, from December 2005 to January 2007. He was also a Professor with Chonnam National University, South Korea, from 2007 to 2020. He is currently a Professor with the School of Computer Science and Engineering, Soongsil University, South Korea. His research interests include computer systems, embedded systems, deep learning, fault diagnosis, and low-power systems.